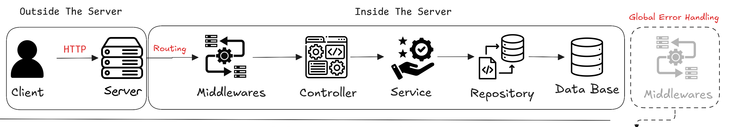
Controllers, Services, Repositories, Middlewares

Why split the code?

Separating your code into Controllers, Services, and Repositories isn't a strict rule you must follow, but it is a highly recommended Design Pattern. It makes your codebase more scalable, maintainable, and much easier to debug as the application grows.

Request Life



Middlewares can operate between different layers of the application, on the use case

Controller/Handler

As the entry point after routing, the Controller receives two main objects from the runtime environment or framework: the Request object and the Response object.

First: Extract Data

Its very first responsibility is to extract data from the incoming request. If it's a GET request, it pulls out the Query Parameters. If it's a POST or PUT request, it extracts the Request Body.

GET
`req.query.age`

POST
`req.body.name`

PUT
`req.params.id`
`req.body`

PATCH
`req.params.id`
`req.body`

DELETE
`req.params.id`

the Controller takes over to send the response. It is responsible for choosing the appropriate HTTP Status Code:

Error Code 400 (Bad Request)

Second: Deserialization

Data travels across the internet in a serialized format, most commonly JSON. For your programming language to actually process this data, it must be deserialized into a native format (like a JavaScript Object, a Python Dictionary, or a Go Struct).

if you are using Node.js, you usually don't do this manually inside the Controller.

Third: Validation & Transformation

Validation

The second major step in the Controller is validating everything coming from the client (path parameters, query parameters, request body). This ensures the data is in the expected format, nothing mandatory is missing, and there is no malicious intent.

Transformation

After validation, we transform the data to make it more convenient for the server to process.

Service

The Service layer manages the flow of the business logic. It can call multiple repository methods, merge their data, trigger external APIs, or send notifications. It coordinates all these tasks and returns the final result to the Controller.

Repository

Interacting with the database (SQL or NoSQL). It constructs queries and returns data.

Single Responsibility

In the Repository layer, each method should do one thing only.

Avoid mixing multiple responsibilities inside the same method

If you need different queries/behaviors, create separate methods (or separate query objects/specifications), each with a clear, specific purpose.

This keeps the Repository:

- cleaner and easier to read
- safer to change
- simpler to test and reuse
- less error-prone during future development

Middlewares Optinal

Middleware is a piece of code that runs in the middle of your backend request lifecycle—between components like Routing and Controller/Handler, and sometimes even before the response is sent back.

Why do we use Middleware?

Because backend apps have many endpoints, and most requests need some common cross-cutting tasks, such as:

- authentication (verify token/session)
- authorization (check roles/permissions)
- rate limiting
- logging
- CORS / security headers
- global error handling

Without middleware, you'd have to duplicate these checks inside every single controller—making the code messy and hard to maintain.

Where does Middleware run? (In "middle boundaries")

In the request lifecycle, middleware is executed across boundaries between major steps:

- Between Routing and Controller
- Potentially between other steps

How does Middleware execute?

In many frameworks (like Express), middleware receives:

req (the incoming request)
res (the response object)
next (a function to continue execution)

Case 1: Middleware passes the request

If everything is OK, middleware calls: `next()`

Then the request continues to: the next middleware, or the Controller/Handler.

Case 2: Middleware stops and returns a response

If something fails (example: invalid token), middleware returns a response immediately like:

`401 Unauthorized` `403 Forbidden` `429 Too Many Requests`

And it does not call next(), so the request never reaches the controller.

Common Middleware types

1) CORS Middleware

Checks the Origin of the request.

If origin is allowed, it adds headers so the browser permits the request.

If not allowed, the browser blocks it.

2) Security Headers Middleware

Adds security-related headers like Content-Security-Policy, etc.

Often applied on every request.

3) Authentication Middleware

Validates user credentials (token/session).

On failure: returns 401 immediately.

On success: stores user info (userId, role, permissions) in request-scoped storage (request context), then calls `next()`.

4) Authorization Middleware

Checks if authenticated user has permissions for the endpoint. On failure: returns 403 (or your standard).

5) Rate Limiting Middleware

Prevents too many requests in a short time.

If exceeded: returns 429 Too Many Requests.

6) Logging / Monitoring Middleware

Logs request details (path, method, query, duration, status). Helps debugging and auditing.

7) Global Error Handling Middleware

Catches unstructured errors from anywhere in the pipeline (middleware/controller/service).

Converts them into a consistent structured error response